

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

отчет по блоку задач №3 (Яндекс.Контест)

Выполнил:

Студент группы Р3210

Бармичев Григорий Андреевич

Преподаватель:

Косяков Михаил Сергеевич

Санкт-Петербург, 2026г.

Задача №9 «I. Машинки»

Описание:

Дана последовательность из P запросов к машинкам. На полу одновременно может находиться не более K машинок, а если нужной машинки нет, мама должна достать её с полки. Нужно минимизировать число таких операций.

Доказательство:

Для решения заранее для каждой машинки сохраняем все моменты её будущих появлений. Далее обрабатываем запросы слева направо: если машинка уже на полу, обновляем информацию о её следующем использовании, иначе выполняем операцию. Если места нет, убираем с пола ту машинку, которая понадобится позже всех. Для этого используется очередь с приоритетом.

Если нужной машинки нет на полу, операцию всё равно придётся делать. Значит, важно только правильно выбрать, какую машинку убрать, когда пол уже заполнен.

Пусть среди машинок на полу есть такая, которая понадобится позже всех. Если убрать не её, а другую машинку, которая понадобится раньше, то эта другая машинка потребуется раньше и маме придётся выполнять лишнюю операцию раньше, чем это было бы нужно. Следовательно, выгоднее всегда убирать машинку с самым поздним следующим использованием.

Алгоритмическая сложность:

По времени: сначала за $O(P)$ строятся списки будущих появлений, затем каждый запрос обрабатывается через очередь с приоритетом за $O(\log P)$. Временная сложность равна $O(P \log P)$.

По памяти: хранятся история запросов, списки будущих появлений, массив присутствия машинок на полу и очередь с приоритетом. Пространственная сложность равна $O(N + P)$.

Код:

```
#include <algorithm>
#include <iostream>
#include <queue>
#include <vector>

using namespace std;

struct CarState {
    int next_pos;
    int id;

    bool operator<(const CarState& other) const {
        return next_pos < other.next_pos;
    }
};

int main() {
    int n, k, p;
    if (!(cin >> n >> k >> p)) {
        return 0;
    }

    vector<int> history(p);
```

```

vector<vector<int>> future_indices(n + 1);

for (int i = 0; i < p; ++i) {
    cin >> history[i];
    future_indices[history[i]].push_back(i);
}

for (int i = 1; i <= n; ++i) {
    future_indices[i].push_back(1000001);
    reverse(future_indices[i].begin(), future_indices[i].end());
}

vector<bool> active(n + 1, false);
priority_queue<CarState> heap;
int ans = 0;
int floor_count = 0;

for (int i = 0; i < p; ++i) {
    int current_car = history[i];

    future_indices[current_car].pop_back();
    int next_time = future_indices[current_car].back();

    if (active[current_car]) {
        heap.push({next_time, current_car});
        continue;
    }

    ans++;
    if (floor_count < k) {
        floor_count++;
    } else {
        while (!heap.empty()) {
            CarState top = heap.top();
            heap.pop();

            if (active[top.id] && future_indices[top.id].back() == top.next_pos) {
                active[top.id] = false;
                break;
            }
        }
    }

    active[current_car] = true;
    heap.push({next_time, current_car});
}

cout << ans << endl;

return 0;
}

```

Задача №10 «J. Гоблины и очереди»

Описание:

Дана очередь гоблинов, для которой нужно обрабатывать три типа запросов: обычный гoblin встает в конец очереди, привилегированный - в середину, а по запросу - первый гoblin уходит к шаманам. Требуется для каждого запроса входа вывести номер гоблина, который должен уйти.

Доказательство:

Очередь хранится в двух деках: head и tail, так что вся очередь равна head + tail. При добавлении обычного гоблина он помещается в конец tail, а привилегированного - в начало tail, что как раз соответствует вставке в середину. После каждой операции выполняется балансировка, которая восстанавливает нужное соотношение размеров двух деков и не нарушает порядок элементов. Таким образом очередь всегда отражает корректно, при запросе входа просто берётся 1 элемент head.

Алгоритмическая сложность:

По времени: каждая операция добавления, удаления и балансировки в деке выполняется за константное время, поэтому все N запросов обрабатываются за $O(N)$.

По памяти: в двух деках суммарно хранится не более N гоблинов, поэтому пространственная сложность равна $O(N)$.

Код:

```
#include <deque>
#include <iostream>

using namespace std;

struct GoblinQueue {
    deque<int> head;
    deque<int> tail;

    void balance() {
        if (head.size() < tail.size()) {
            head.push_back(tail.front());
            tail.pop_front();
        } else if (head.size() > tail.size() + 1) {
            tail.push_front(head.back());
            head.pop_back();
        }
    }

    void push_end(int id) {
        tail.push_back(id);
        balance();
    }

    void push_mid(int id) {
        tail.push_front(id);
        balance();
    }

    int pop_first() {
        int res = head.front();
```

```

    head.pop_front();
    balance();
    return res;
}
};

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    if (!(cin >> n)) {
        return 0;
    }

    GoblinQueue q;

    for (int i = 0; i < n; ++i) {
        char op;
        cin >> op;

        if (op == '-') {
            cout << q.pop_first() << "\n";
        } else {
            int id;
            cin >> id;
            if (op == '+') {
                q.push_end(id);
            } else {
                q.push_mid(id);
            }
        }
    }

    return 0;
}

```

Задача №11 «К. Менеджер памяти-1»

Описание:

Дан массив из N ячеек памяти и M запросов двух типов: выделить блок из K подряд идущих свободных ячеек или освободить память, выделенную по запросу с номером T . При успешном выделении нужно выдать самый левый подходящий блок, то есть такой, перед первой ячейкой которого нет свободной ячейки.

Доказательство:

В декартовом дереве хранятся все максимальные свободные отрезки памяти. Для каждой вершины дополнительно поддерживается mx - максимальная длина свободного отрезка в её поддереве.

Далее мы ищем самый левый отрезок длины не меньше K : если подходящий есть в левом поддереве, идём туда, иначе проверяем текущую вершину, иначе идём вправо.

При освобождении памяти отрезок восстанавливается. Если он примыкает к свободному отрезку слева или справа, они сливаются в один более длинный свободный отрезок. После этого в дереве снова хранятся все максимальные свободные отрезки.

Алгоритмическая сложность:

По времени: поиск подходящего блока, разбиение, слияние, вставка и удаление в декартовом дереве в среднем выполняются за $O(\log M)$. Поэтому все M запросов обрабатываются за $O(M \log M)$.

По памяти: хранятся сведения о запросах и вершины декартового дерева, число которых не более $O(M)$. Пространственная сложность равна $O(M)$

Код:

```
#include <algorithm>
#include <iostream>
#include <random>
#include <vector>

using namespace std;

struct Node {
    long long l;
    long long r;
    long long mx;
    unsigned int priority;
    int left;
    int right;
};

Node pool[300005];
int node_ptr = 0;

int create_node(long long l, long long r, unsigned int priority) {
    node_ptr++;
    pool[node_ptr].l = l;
    pool[node_ptr].r = r;
    pool[node_ptr].mx = r - l + 1;
    pool[node_ptr].priority = priority;
    pool[node_ptr].left = 0;
    pool[node_ptr].right = 0;
    return node_ptr;
}

long long get_max_val(int t) {
    if (t == 0) {
        return 0;
    }
    return pool[t].mx;
}

void update(int t) {
    if (t == 0) {
        return;
    }
}
```

```

}
long long current_len = pool[t].r - pool[t].l + 1;
long long left_mx = get_max_val(pool[t].left);
long long right_mx = get_max_val(pool[t].right);
pool[t].mx = max({current_len, left_mx, right_mx});
}

```

```

void split(int t, long long key, int& a, int& b) {
    if (t == 0) {
        a = 0;
        b = 0;
        return;
    }
    if (pool[t].l < key) {
        split(pool[t].right, key, pool[t].right, b);
        a = t;
    } else {
        split(pool[t].left, key, a, pool[t].left);
        b = t;
    }
    update(t);
}

```

```

int merge(int a, int b) {
    if (a == 0) {
        return b;
    }
    if (b == 0) {
        return a;
    }
    if (pool[a].priority > pool[b].priority) {
        pool[a].right = merge(pool[a].right, b);
        update(a);
        return a;
    } else {
        pool[b].left = merge(a, pool[b].left);
        update(b);
        return b;
    }
}

```

```

int find_leftmost(int t, long long size) {
    if (t == 0 || pool[t].mx < size) {
        return 0;
    }
    if (pool[t].left != 0) {
        if (pool[pool[t].left].mx >= size) {
            return find_leftmost(pool[t].left, size);
        }
    }
    if (pool[t].r - pool[t].l + 1 >= size) {
        return t;
    }
    return find_leftmost(pool[t].right, size);
}

```

```

}

int main() {
    long long n;
    int m;
    if (!(cin >> n >> m)) {
        return 0;
    }

    mt19937 rng(1337);
    int root = create_node(1, n, rng());

    vector<long long> alloc_start(m + 1, -1);
    vector<long long> alloc_len(m + 1, 0);

    for (int i = 1; i <= m; ++i) {
        long long query;
        cin >> query;

        if (query > 0) {
            int block_id = find_leftmost(root, query);
            if (block_id == 0) {
                cout << "-1\n";
                alloc_start[i] = -1;
            } else {
                long long b_l = pool[block_id].l;
                long long b_r = pool[block_id].r;
                cout << b_l << "\n";
                alloc_start[i] = b_l;
                alloc_len[i] = query;

                int t1, t2, t3;
                split(root, b_l, t1, t2);
                split(t2, b_l + 1, t2, t3);
                root = merge(t1, t3);

                if (b_l + query <= b_r) {
                    int remainder = create_node(b_l + query, b_r, rng());
                    int l_part, r_part;
                    split(root, b_l + query, l_part, r_part);
                    root = merge(merge(l_part, remainder), r_part);
                }
            }
        } else {
            int target_id = (int)(-query);
            if (alloc_start[target_id] != -1) {
                long long l = alloc_start[target_id];
                long long r = l + alloc_len[target_id] - 1;

                int prv_id = 0;
                int curr = root;
                while (curr != 0) {
                    if (pool[curr].l < l) {
                        prv_id = curr;
                    }
                }
            }
        }
    }
}

```



```

    curr = pool[curr].right;
} else {
    curr = pool[curr].left;
}
}
if (prv_id != 0) {
    if (pool[prv_id].r + 1 == l) {
        l = pool[prv_id].l;
        int t1, t2, t3;
        split(root, pool[prv_id].l, t1, t2);
        split(t2, pool[prv_id].l + 1, t2, t3);
        root = merge(t1, t3);
    }
}

int nxt_id = 0;
curr = root;
while (curr != 0) {
    if (pool[curr].l > r) {
        nxt_id = curr;
        curr = pool[curr].left;
    } else {
        curr = pool[curr].right;
    }
}
if (nxt_id != 0) {
    if (r + 1 == pool[nxt_id].l) {
        r = pool[nxt_id].r;
        int t1, t2, t3;
        split(root, pool[nxt_id].l, t1, t2);
        split(t2, pool[nxt_id].l + 1, t2, t3);
        root = merge(t1, t3);
    }
}

int merged = create_node(l, r, rng());
int l_part, r_part;
split(root, l, l_part, r_part);
root = merge(merge(l_part, merged), r_part);
alloc_start[target_id] = -1;
}
}
}

return 0;
}

```

Задача №12 «L. Минимум на отрезке»

Описание:

Дана последовательность из N чисел и окно длины K , которое сдвигается вправо на 1 позицию. Для каждого положения окна требуется найти минимум среди элементов, попавших в это окно.

Доказательство:

В решении поддерживается монотонная очередь из индексов элементов текущего окна. Перед добавлением нового элемента из конца очереди удаляются все индексы элементов, не меньших нового, потому что они уже не смогут стать минимумом ни в текущем окне, ни в следующих. Из начала очереди удаляются индексы элементов, которые вышли за левую границу окна. После этого в очереди остаются только индексы элементов текущего окна, причём их значения упорядочены по возрастанию. Поэтому первый индекс в очереди всегда соответствует минимальному элементу окна.

Алгоритмическая сложность:

По времени: каждый элемент добавляется в очередь один раз и удаляется из неё не более одного раза. Временная сложность равна $O(N)$.

По памяти: хранится массив чисел и очередь индексов размера не более K . Пространственная сложность равна $O(N)$.

Код:

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    int n, k;
    if (!(cin >> n >> k)) {
        return 0;
    }

    vector<int> numbers(n);
    for (int i = 0; i < n; ++i) {
        cin >> numbers[i];
    }

    vector<int> q_indices(n);
    int head = 0;
    int tail = 0;

    for (int i = 0; i < n; ++i) {
        if (head < tail && q_indices[head] <= i - k) {
            head++;
        }

        while (head < tail && numbers[q_indices[tail - 1]] >= numbers[i]) {
            tail--;
        }

        q_indices[tail] = i;
        tail++;

        if (i >= k - 1) {
            cout << numbers[q_indices[head]] << " ";
        }
    }
}
```

```
return 0;  
}
```